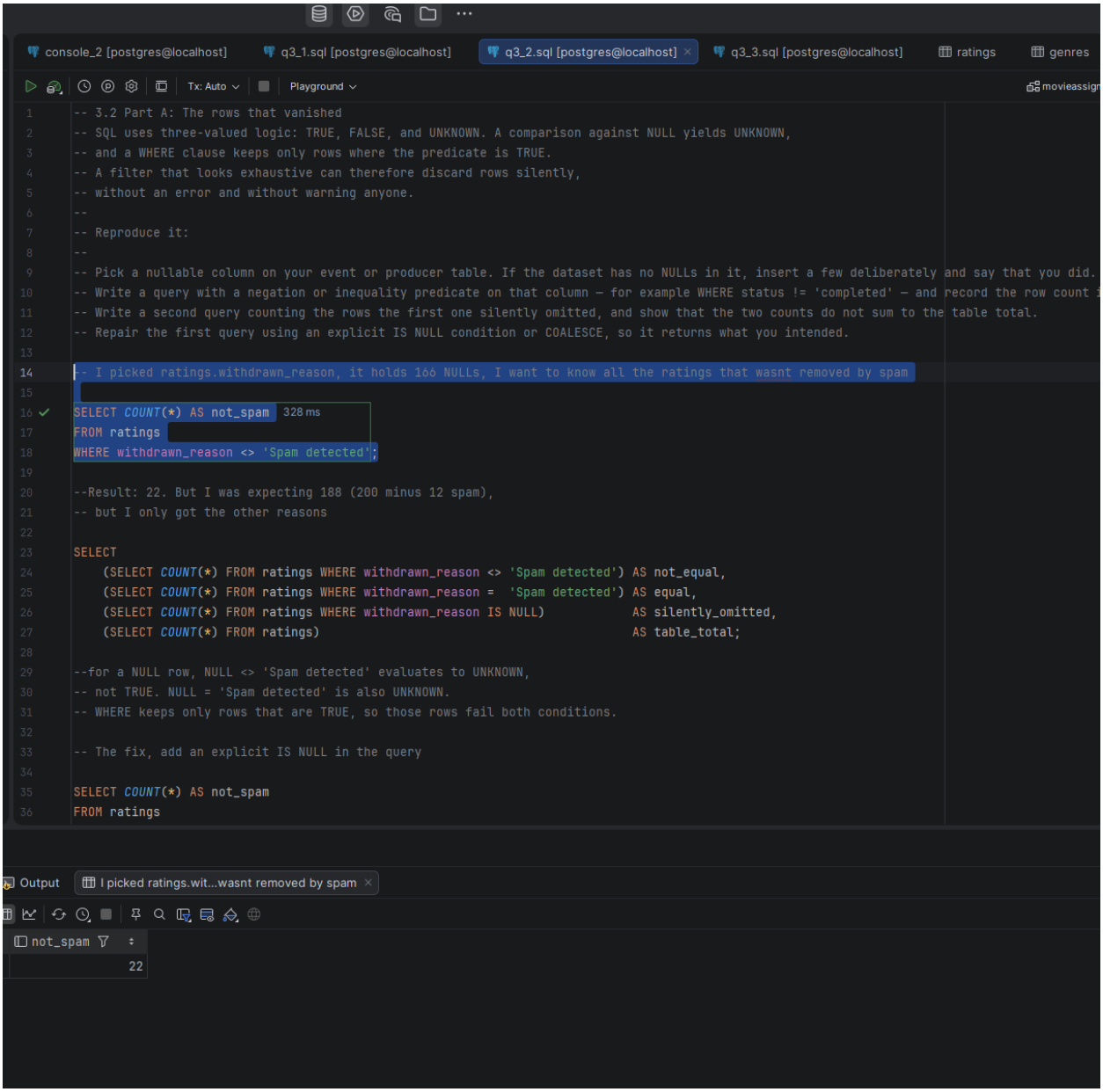
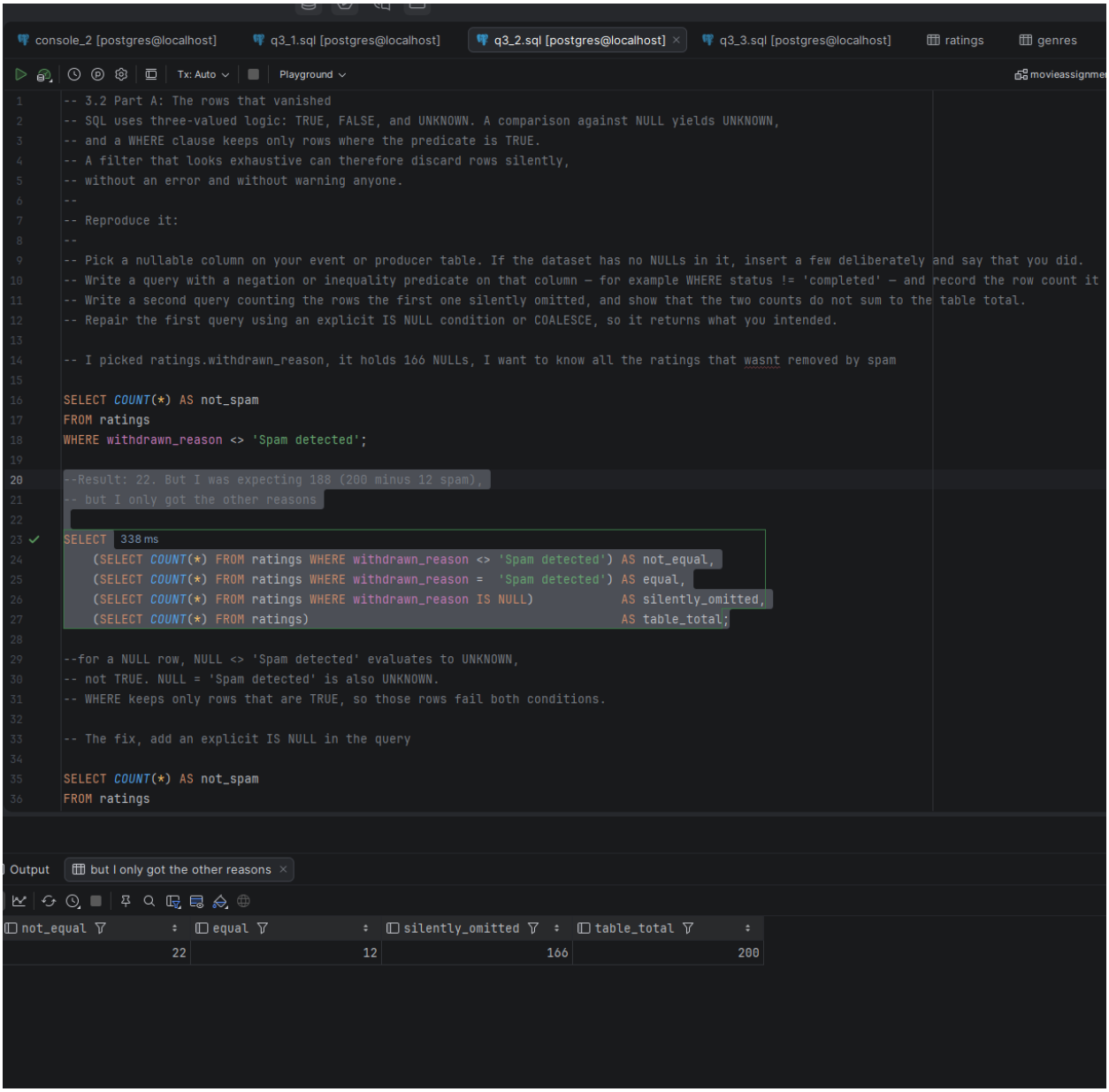
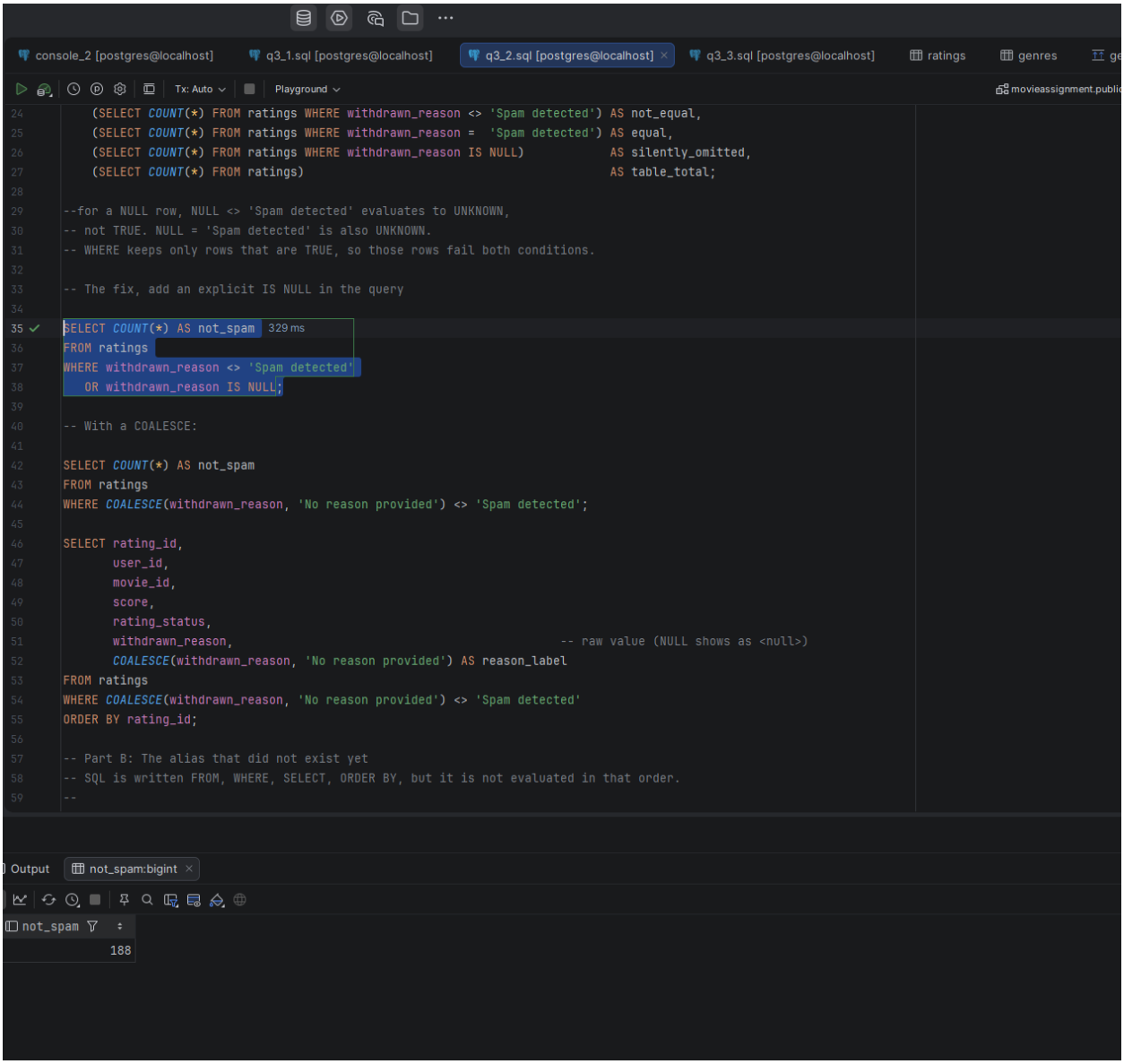
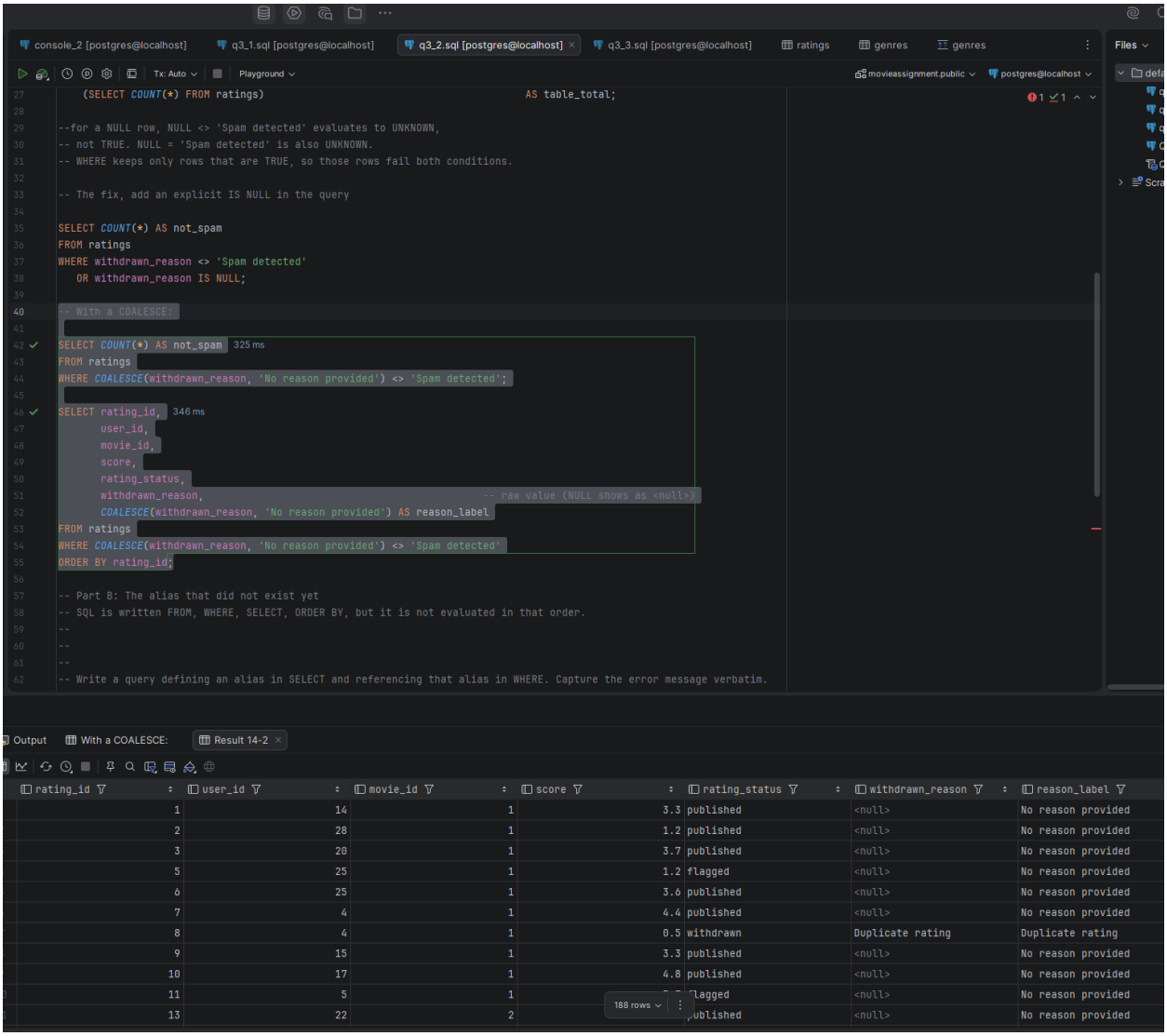
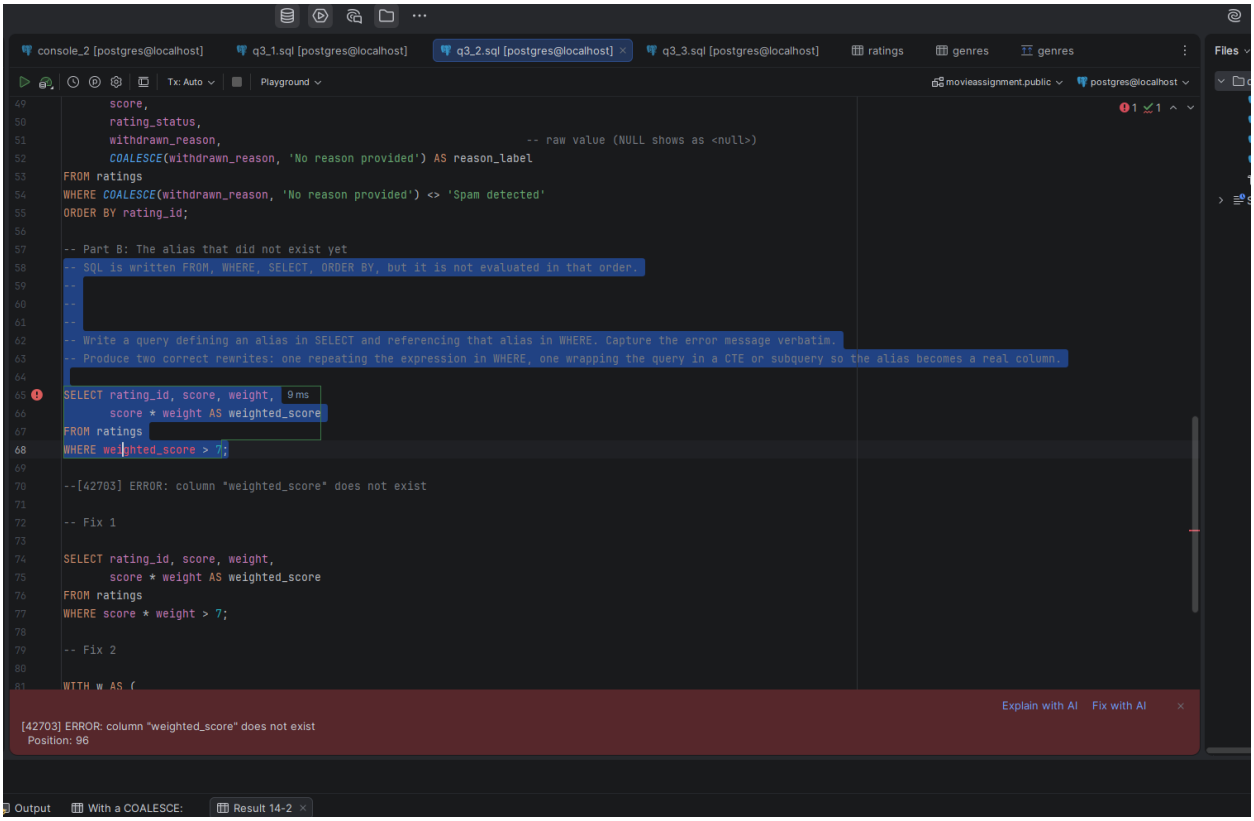
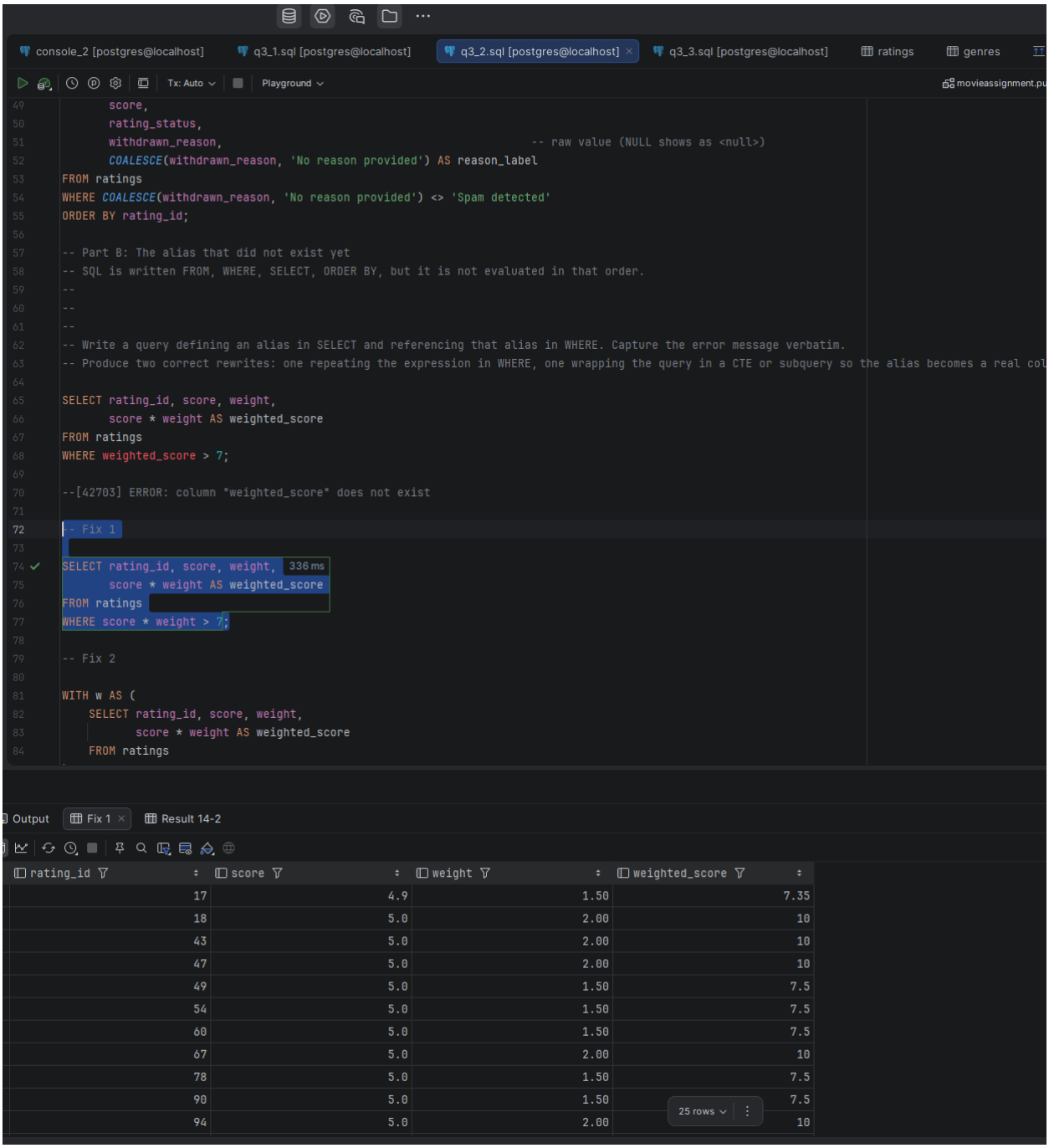

console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
ratings
genres
Tx: Auto
Playground
movieassign
-- 3.2 Part A: The rows that vanished
-- SQL uses three-valued logic: TRUE, FALSE, and UNKNOWN. A comparison against NULL yields UNKNOWN,
-- and a WHERE clause keeps only rows where the predicate is TRUE.
-- A filter that looks exhaustive can therefore discard rows silently,
-- without an error and without warning anyone.
--
-- Reproduce it:
--
-- Pick a nullable column on your event or producer table. If the dataset has no NULLs in it, insert a few deliberately and say that you did.
-- Write a query with a negation or inequality predicate on that column — for example WHERE status != 'completed' — and record the row count i
-- Write a second query counting the rows the first one silently omitted, and show that the two counts do not sum to the table total.
-- Repair the first query using an explicit IS NULL condition or COALESCE, so it returns what you intended.
-- I picked ratings.withdrawn_reason, it holds 166 NULLs, I want to know all the ratings that wasnt removed by spam
SELECT COUNT(*) AS not_spam
328 ms
FROM ratings
WHERE withdrawn_reason <> 'Spam detected';
--Result: 22. But I was expecting 188 (200 minus 12 spam),
-- but I only got the other reasons
SELECT
(SELECT COUNT(*) FROM ratings WHERE withdrawn_reason <> 'Spam detected') AS not_equal,
(SELECT COUNT(*) FROM ratings WHERE withdrawn_reason = 'Spam detected') AS equal,
(SELECT COUNT(*) FROM ratings WHERE withdrawn_reason IS NULL) AS silently_omitted,
(SELECT COUNT(*) FROM ratings) AS table_total;
--for a NULL row, NULL <> 'Spam detected' evaluates to UNKNOWN,
-- not TRUE. NULL = 'Spam detected' is also UNKNOWN.
-- WHERE keeps only rows that are TRUE, so those rows fail both conditions.
-- The fix, add an explicit IS NULL in the query
SELECT COUNT(*) AS not_spam
FROM ratings
Output
I picked ratings.wit...wasnt removed by spam
not_spam
22

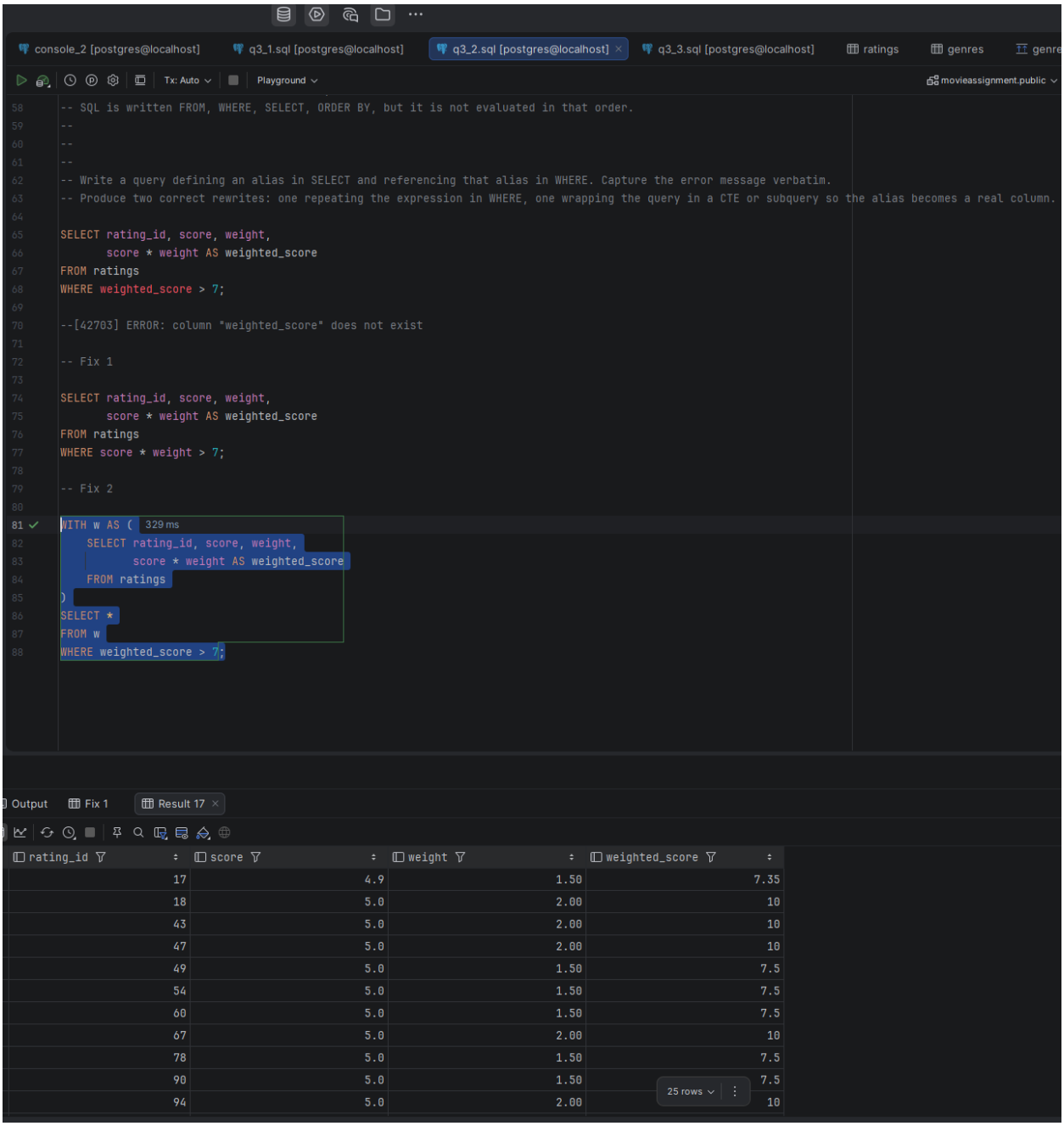

console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
ratings
genres
Tx: Auto
Playground
movieassignme
-- 3.2 Part A: The rows that vanished
-- SQL uses three-valued logic: TRUE, FALSE, and UNKNOWN. A comparison against NULL yields UNKNOWN,
-- and a WHERE clause keeps only rows where the predicate is TRUE.
-- A filter that looks exhaustive can therefore discard rows silently,
-- without an error and without warning anyone.
--
-- Reproduce it:
--
-- Pick a nullable column on your event or producer table. If the dataset has no NULLs in it, insert a few deliberately and say that you did.
-- Write a query with a negation or inequality predicate on that column — for example WHERE status != 'completed' — and record the row count it
-- Write a second query counting the rows the first one silently omitted, and show that the two counts do not sum to the table total.
-- Repair the first query using an explicit IS NULL condition or COALESCE, so it returns what you intended.
-- I picked ratings.withdrawn_reason, it holds 166 NULLs, I want to know all the ratings that wasnt removed by spam
SELECT COUNT(*) AS not_spam
FROM ratings
WHERE withdrawn_reason <> 'Spam detected';
--Result: 22. But I was expecting 188 (200 minus 12 spam),
-- but I only got the other reasons
SELECT 338 ms
(SELECT COUNT(*) FROM ratings WHERE withdrawn_reason <> 'Spam detected') AS not_equal,
(SELECT COUNT(*) FROM ratings WHERE withdrawn_reason = 'Spam detected') AS equal,
(SELECT COUNT(*) FROM ratings WHERE withdrawn_reason IS NULL) AS silently_omitted,
(SELECT COUNT(*) FROM ratings) AS table_total;
--for a NULL row, NULL <> 'Spam detected' evaluates to UNKNOWN,
-- not TRUE. NULL = 'Spam detected' is also UNKNOWN.
-- WHERE keeps only rows that are TRUE, so those rows fail both conditions.
-- The fix, add an explicit IS NULL in the query
SELECT COUNT(*) AS not_spam
FROM ratings
Output
but I only got the other reasons
not_equal
equal
silently_omitted
table_total
22
12
166
200


console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
ratings
genres
Tx: Auto
Playground
movieassignment.public
(SELECT COUNT(*) FROM ratings WHERE withdrawn_reason <> 'Spam detected') AS not_equal,
(SELECT COUNT(*) FROM ratings WHERE withdrawn_reason = 'Spam detected') AS equal,
(SELECT COUNT(*) FROM ratings WHERE withdrawn_reason IS NULL) AS silently_omitted,
(SELECT COUNT(*) FROM ratings) AS table_total;
--for a NULL row, NULL <> 'Spam detected' evaluates to UNKNOWN,
-- not TRUE. NULL = 'Spam detected' is also UNKNOWN.
-- WHERE keeps only rows that are TRUE, so those rows fail both conditions.
-- The fix, add an explicit IS NULL in the query
SELECT COUNT(*) AS not_spam
329 ms
FROM ratings
WHERE withdrawn_reason <> 'Spam detected'
OR withdrawn_reason IS NULL;
-- With a COALESCE:
SELECT COUNT(*) AS not_spam
FROM ratings
WHERE COALESCE(withdrawn_reason, 'No reason provided') <> 'Spam detected';
SELECT rating_id,
user_id,
movie_id,
score,
rating_status,
withdrawn_reason, -- raw value (NULL shows as <null>)
COALESCE(withdrawn_reason, 'No reason provided') AS reason_label
FROM ratings
WHERE COALESCE(withdrawn_reason, 'No reason provided') <> 'Spam detected'
ORDER BY rating_id;
-- Part B: The alias that did not exist yet
-- SQL is written FROM, WHERE, SELECT, ORDER BY, but it is not evaluated in that order.
--
Output
not_spam:bigint
not_spam
188


console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
ratings
genres
genres
Files
Tx: Auto
Playground
movieassignment.public
postgres@localhost
(SELECT COUNT(*) FROM ratings) AS table_total;
--for a NULL row, NULL <> 'Spam detected' evaluates to UNKNOWN,
-- not TRUE. NULL = 'Spam detected' is also UNKNOWN.
-- WHERE keeps only rows that are TRUE, so those rows fail both conditions.
-- The fix, add an explicit IS NULL in the query
SELECT COUNT(*) AS not_spam
FROM ratings
WHERE withdrawn_reason <> 'Spam detected'
OR withdrawn_reason IS NULL;
-- With a COALESCE:
SELECT COUNT(*) AS not_spam 325 ms
FROM ratings
WHERE COALESCE(withdrawn_reason, 'No reason provided') <> 'Spam detected';
SELECT rating_id, 346 ms
user_id,
movie_id,
score,
rating_status,
withdrawn_reason, -- raw value (NULL shows as <null>)
COALESCE(withdrawn_reason, 'No reason provided') AS reason_label
FROM ratings
WHERE COALESCE(withdrawn_reason, 'No reason provided') <> 'Spam detected'
ORDER BY rating_id;
-- Part B: The alias that did not exist yet
-- SQL is written FROM, WHERE, SELECT, ORDER BY, but it is not evaluated in that order.
--
--
--
-- Write a query defining an alias in SELECT and referencing that alias in WHERE. Capture the error message verbatim.
Output
With a COALESCE:
Result 14-2
rating_id
user_id
movie_id
score
rating_status
withdrawn_reason
reason_label
1 14 1 3.3 published <null> No reason provided
2 28 1 1.2 published <null> No reason provided
3 20 1 3.7 published <null> No reason provided
5 25 1 1.2 flagged <null> No reason provided
6 25 1 3.6 published <null> No reason provided
7 4 1 4.4 published <null> No reason provided
8 4 1 0.5 withdrawn Duplicate rating Duplicate rating
9 15 1 3.3 published <null> No reason provided
10 17 1 4.8 published <null> No reason provided
11 5 1 flagged <null> No reason provided
13 22 2 published <null> No reason provided
188 rows


console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
ratings
genres
genres
Files
Tx: Auto
Playground
movieassignment.public
postgres@localhost
score,
rating_status,
withdrawn_reason, -- raw value (NULL shows as <null>)
COALESCE(withdrawn_reason, 'No reason provided') AS reason_label
FROM ratings
WHERE COALESCE(withdrawn_reason, 'No reason provided') <> 'Spam detected'
ORDER BY rating_id;
-- Part B: The alias that did not exist yet
-- SQL is written FROM, WHERE, SELECT, ORDER BY, but it is not evaluated in that order.
--
--
--
-- Write a query defining an alias in SELECT and referencing that alias in WHERE. Capture the error message verbatim.
-- Produce two correct rewrites: one repeating the expression in WHERE, one wrapping the query in a CTE or subquery so the alias becomes a real column.
SELECT rating_id, score, weight, 9 ms
score * weight AS weighted_score
FROM ratings
WHERE weighted_score > 7;
--[42703] ERROR: column "weighted_score" does not exist
-- Fix 1
SELECT rating_id, score, weight,
score * weight AS weighted_score
FROM ratings
WHERE score * weight > 7;
-- Fix 2
WITH w AS (
Explain with AI
Fix with AI
[42703] ERROR: column "weighted_score" does not exist
Position: 96
Output
With a COALESCE:
Result 14-2


console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
ratings
genres
Tx: Auto
Playground
movieassignment.pu
score,
rating_status,
withdrawn_reason, -- raw value (NULL shows as <null>)
COALESCE(withdrawn_reason, 'No reason provided') AS reason_label
FROM ratings
WHERE COALESCE(withdrawn_reason, 'No reason provided') <> 'Spam detected'
ORDER BY rating_id;
-- Part B: The alias that did not exist yet
-- SQL is written FROM, WHERE, SELECT, ORDER BY, but it is not evaluated in that order.
--
--
--
-- Write a query defining an alias in SELECT and referencing that alias in WHERE. Capture the error message verbatim.
-- Produce two correct rewrites: one repeating the expression in WHERE, one wrapping the query in a CTE or subquery so the alias becomes a real col
SELECT rating_id, score, weight,
score * weight AS weighted_score
FROM ratings
WHERE weighted_score > 7;
--[42703] ERROR: column "weighted_score" does not exist
-- Fix 1
SELECT rating_id, score, weight, 336 ms
score * weight AS weighted_score
FROM ratings
WHERE score * weight > 7;
-- Fix 2
WITH w AS (
SELECT rating_id, score, weight,
score * weight AS weighted_score
FROM ratings
Output
Fix 1
Result 14-2
rating_id
score
weight
weighted_score
17 4.9 1.50 7.35
18 5.0 2.00 10
43 5.0 2.00 10
47 5.0 2.00 10
49 5.0 1.50 7.5
54 5.0 1.50 7.5
60 5.0 1.50 7.5
67 5.0 2.00 10
78 5.0 1.50 7.5
90 5.0 1.50 7.5
94 5.0 2.00 10
25 rows


console_2 [postgres@localhost]
q3_1.sql [postgres@localhost]
q3_2.sql [postgres@localhost]
q3_3.sql [postgres@localhost]
ratings
genres
Tx: Auto
Playground
movieassignment.public
-- SQL is written FROM, WHERE, SELECT, ORDER BY, but it is not evaluated in that order.
--
--
--
-- Write a query defining an alias in SELECT and referencing that alias in WHERE. Capture the error message verbatim.
-- Produce two correct rewrites: one repeating the expression in WHERE, one wrapping the query in a CTE or subquery so the alias becomes a real column.
SELECT rating_id, score, weight,
score * weight AS weighted_score
FROM ratings
WHERE weighted_score > 7;
--[42703] ERROR: column "weighted_score" does not exist
-- Fix 1
SELECT rating_id, score, weight,
score * weight AS weighted_score
FROM ratings
WHERE score * weight > 7;
-- Fix 2
WITH w AS (329 ms
SELECT rating_id, score, weight,
score * weight AS weighted_score
FROM ratings
)
SELECT *
FROM w
WHERE weighted_score > 7;
Output
Fix 1
Result 17
rating_id
score
weight
weighted_score
17 4.9 1.50 7.35
18 5.0 2.00 10
43 5.0 2.00 10
47 5.0 2.00 10
49 5.0 1.50 7.5
54 5.0 1.50 7.5
60 5.0 1.50 7.5
67 5.0 2.00 10
78 5.0 1.50 7.5
90 5.0 1.50 7.5
94 5.0 2.00 10
25 rows